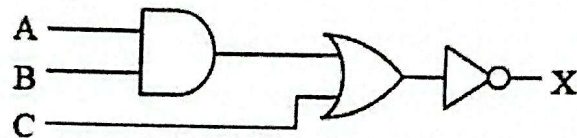


2: Boolean Algebra

2.1 Need for Boolean Expressions

At this point in our study of digital circuits, we have two methods for representing combinational logic: schematics and truth tables.

A	B	C	X
0	0	0	1
0	0	1	0
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	0



Truth Table and Schematic of Combinational Logic

These two methods are inadequate for a number of reasons:

- Both schematics and truth tables take too much space to describe the operation of complex circuits with numerous inputs.
- The truth table "hides" circuit information.
- The schematic diagram is difficult to use when trying to determine output values for each input combination.

To overcome these problems, a discipline much like algebra is practiced that uses expressions to describe digital circuitry. These expressions, which are called **Boolean expressions**, use the input variable names, A, B, C, etc., and combine them using symbols representing the AND, OR, and NOT gates. These Boolean expressions can be used to describe or evaluate the output of a circuit.

A set of rules exist that when applied to Boolean expressions can simplify them. A simpler expression that produces the same output can be realized with fewer logic gates. A lower gate count results in cheaper circuitry, smaller circuit boards, and lower power consumption. Thus lowering the cost, increasing the speed, and use of lesser memory.

Symbols of Boolean Algebra

The following expressions hold true in math.

$$0 \cdot 0 = 0, 0 \cdot 1 = 0, 1 \cdot 0 = 0, 1 \cdot 1 = 1$$

This looks like the AND function allowing an analogy to be drawn between the mathematical multiply and the Boolean AND functions. In Boolean algebra, A AND'ed with B is written $A \cdot B$.

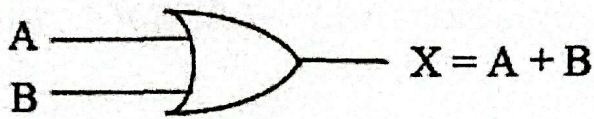


Mathematical addition has a similar parallel in Boolean algebra. The following four mathematical expressions hold true for addition.

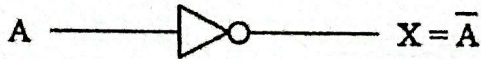
$$0 + 0 = 0, 0 + 1 = 1, 1 + 0 = 1, 1 + 1 = 2$$

The first three operations match the OR function, and if the last operation is viewed as having a non-zero result instead of the decimal result of two, it too can be viewed as operating similar to the OR function. Therefore, the Boolean OR function is analogous to the mathematical function of addition.

2: Boolean Algebra



The NOT is represented with a bar across the inverted element.



The NOT operation may be used to invert the result of a larger expression. For example, the NAND function which places an inverter at the output of an AND gate is written as:

$$X = \overline{A \cdot B}$$

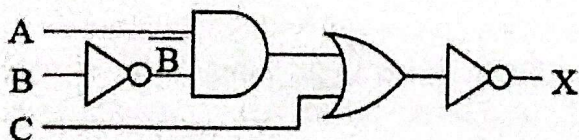
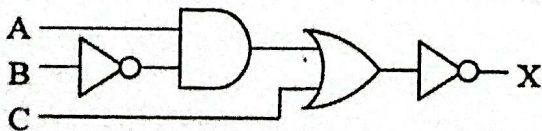
Since the bar goes across $A \cdot B$, the NOT is performed after the AND.

The two-input XOR operation is represented using the symbol $\oplus$, but it can also be represented using a Boolean expression. Basically, the two-input XOR equals one if $A = 0$ and $B = 1$ or if $A = 1$ and $B = 0$. This gives us the following expression.

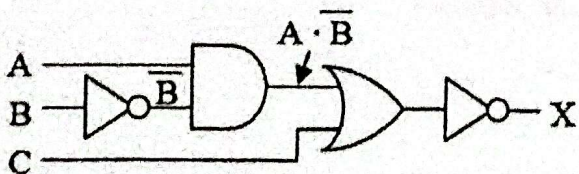
$$X = A \oplus B = \bar{A} \cdot B + A \cdot \bar{B}$$

Boolean Expressions of Combinational Logic

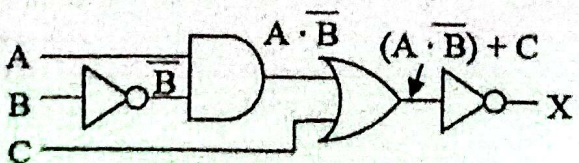
Just as mathematical algebra combines multiplication and addition to create complex expressions, Boolean algebra combines AND, OR, and NOT functions to represent complex combinational logic. Take for example the circuit shown below.



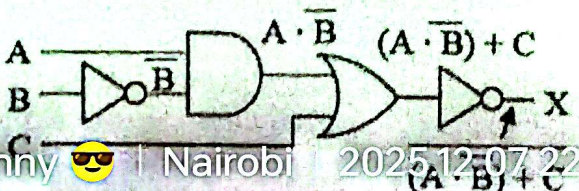
a) B goes through the first inverter which outputs a $\bar{B}$



b) A and $\bar{B}$ go through the AND gate which outputs $A \cdot \bar{B}$.



c) $A \cdot \bar{B}$ and C go through the OR gate which outputs $(A \cdot \bar{B}) + C$.

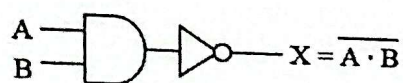
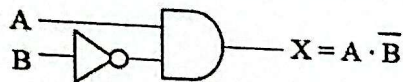


d) The output of the OR gate goes through a second inverter giving us our result.

2: Boolean Algebra

Notice the use of parenthesis in step c. Just as in mathematical algebra, parenthesis can be used to force the order in which operations are taken. In the absence of parenthesis, however, the AND, OR, and NOT functions have an order of precedence.

To begin with, AND takes precedence over OR unless overridden by parenthesis. NOT is a special case in that it can act like a set of parenthesis. If the bar indicating the NOT function spans a single variable, it takes precedence over AND and OR. If, however, the NOT bar spans an expression, the expression beneath the bar must be evaluated before the NOT is taken. The figure below presents two examples of handling precedence with the NOT function.



Understanding this is vital because unlike the mathematical inverse, the two expressions below are *not* equivalent.

$$\overline{A \cdot B} \neq \overline{A} \cdot \overline{B}$$

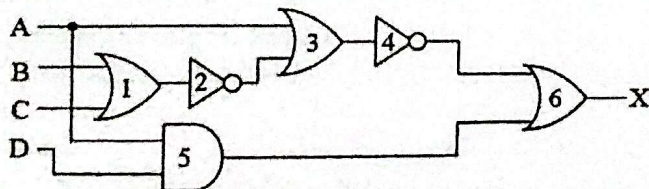
For example, using parenthesis and the order of precedence, the Boolean expression below has a single interpretation.

$$X = A \cdot D + \overline{(A + B + C)}$$

The following steps show the order to evaluate the above expression.

1. OR B with C because the operation is contained under a single NOT bar and is contained within the lowest set of parenthesis
2. Invert the result of step 1 because NOT takes precedence over OR
3. OR A with the result of step 2 because of the parenthesis
4. Invert result of step 3
5. AND A and D because AND takes precedence over OR
6. OR the results of steps 4 and 5

We can use this order of operations to convert the expression to its schematic representation. By starting with a list of inputs to the circuit, then passing each input through the correct gates, we can develop the circuit. The figure below does just this for the previous Boolean expression. We list the inputs for the expression, A, B, C, and D, on the left side of the figure. These inputs are then passed through the gates using the same order as the steps shown above. The number inside each gate of the figure corresponds to the order of the steps.



Laws of Boolean Algebra

The manipulation of algebraic expressions is based on fundamental laws. Some of these laws extend to the manipulation of Boolean expressions. For example, the commutative law of algebra which states that the result of an operation is the same regardless of the order of operands holds true for Boolean algebra too. This is shown for the OR function applied to two variables in the truth tables below.

2: Boolean Algebra

A	B	A + B
0	0	0+0=0
0	1	0+1=1
1	0	1+0=1
1	1	1+1=1

A	B	B + A
0	0	0+0=0
0	1	1+0=1
1	0	0+1=1
1	1	1+1=1

Commutative Law: The results of the Boolean operations AND and OR are the same regardless of the order of their operands.

$$A + B = B + A$$

$$A \cdot B = B \cdot A$$

Associative Law: The results of the Boolean operations AND and OR with three or more operands are the same regardless of which pair of elements are operated on first.

$$A + (B + C) = (A + B) + C$$

$$A \cdot (B \cdot C) = (A \cdot B) \cdot C$$

Distributive Law: The AND'ing of an operand with an OR expression is equivalent to OR'ing the results of an AND between the first operand and each operand within the OR expression.

$$A \cdot (B + C) = A \cdot B + A \cdot C$$

Rules of Boolean Algebra

NOT Rule

If the Boolean inverse of a Boolean inverse is taken, the original value results.

$$\overline{\overline{A}} = A$$



Schematic Form of NOT Rule

OR Rules

If an input to a logic gate is a constant 0 or 1 or if the same signal is connected to more than one input of a gate, a simplification of the expression is almost always possible. This is true for the OR gate as is shown with the following four rules for simplifying the OR function.

$$\text{Rule: } A + 0 = A$$

$$\text{Rule: } A + 1 = 1$$

$$\text{Rule: } A + A = A$$

$$\text{Rule: } A + \overline{A} = 1$$

AND Rules

Just as with the OR gate, if either of the inputs to an AND gate is a constant (logic 0 or logic 1) or if the two inputs are the same or inverses of each other, a simplification can be performed.

$$\text{Rule: } A \cdot 0 = 0$$

$$\text{Rule: } A \cdot 1 = A$$

$$\text{Rule: } A \cdot A = A$$

$$\text{Rule: } A \cdot \overline{A} = 0$$

XOR Rules

Now let's see what happens when we apply these same input conditions to a two-input XOR gate. Remember that a two-input XOR gate outputs a 1 if its inputs are different and a zero if its inputs are the same.

$$\text{Rule: } A \oplus 0 = A$$

2: Boolean Algebra

Rule: $A \oplus 1 = \bar{A}$

Rule: $A \oplus A = 0$

Rule: $A \oplus \bar{A} = 1$

Derivation of Other Rules

If we combine the NOT, OR, and AND rules with the commutative, associative, and distributive laws, we can derive other rules for Boolean algebra. This can be shown with the following example.

Example

Prove that $A + A \cdot B = A$

$$\begin{aligned} A + A \cdot B &= A \cdot 1 + A \cdot B \text{ (Rule: } A \cdot 1 = A\text{)} \\ &= A \cdot (1 + B) \text{ (Distributive Law)} \\ &= A \cdot (B + 1) \text{ (Commutative Law)} \\ &= A \cdot 1 \text{ (Rule: } A + 1 = 1\text{)} \\ &= A \text{ (Rule: } A \cdot 1 = A\text{)} \end{aligned}$$

Remember also that rules of Boolean algebra can be proven using a truth table.

Example

Prove $(A + B) \cdot (A + C) = A + B \cdot C$

Solution

$$\begin{aligned} (A + B) \cdot (A + C) &= (A + B) \cdot A + (A + B) \cdot C \text{ (Distributive Law)} \\ &= A \cdot A + B \cdot A + A \cdot C + B \cdot C \text{ (Distributive Law)} \\ &= A + B \cdot A + A \cdot C + B \cdot C \text{ (Rule: } A \cdot A = A\text{)} \\ &= A + A \cdot B + A \cdot C + B \cdot C \text{ (Commutative Law)} \\ &= A + A \cdot C + B \cdot C \text{ (Rule: } A + A \cdot B = A\text{)} \\ &= A + B \cdot C \text{ (Rule: } A + A \cdot C = A\text{)} \end{aligned}$$

Simplification

The rules of the previous section are summarized below.

1. $A = A$	9. $A \cdot \bar{A} = 0$
2. $A + 0 = A$	10. $A \oplus 0 = A$
3. $A + 1 = 1$	11. $A \oplus 1 = \bar{A}$
4. $A + A = A$	12. $A \oplus A = 0$
5. $A + \bar{A} = 1$	13. $A \oplus \bar{A} = 1$
6. $A \cdot 0 = 0$	14. $A + A \cdot B = A$
7. $A \cdot 1 = A$	15. $A + \bar{A} \cdot B = A + B$
8. $A \cdot A = A$	16. $(A + B) \cdot (A + C) = A + B \cdot C$

Example

Simplify $(A \cdot B + C)(A \cdot B + D)$

Solution

From the rules of Boolean algebra, we know that $(A + B)(A + C) = A + B \cdot C$.
Substitute $A \cdot B$ for A , C for B , and D for C and we get:

$$(A \cdot B + C)(A \cdot B + D) = A \cdot B + C \cdot D$$

Example

Simplify $(A + B) \cdot (B + B)$

2: Boolean Algebra

Solution

$(A + B) \cdot 1$ Anything OR'ed with its inverse is 1

$(A + B)$ Anything AND'ed with 1 is itself

Example

Simplify $\bar{B} \cdot (A + \bar{A} \cdot B)$

Solution

$\bar{B} \cdot A + \bar{B} \cdot \bar{A} \cdot B$	Distributive Law
$\bar{B} \cdot A + \bar{A} \cdot \bar{B} \cdot B$	Associative Law
$\bar{B} \cdot A + \bar{A} \cdot 0$	Anything AND'ed with its inverse is 0
$\bar{B} \cdot A + 0$	Anything AND'ed with 0 is 0
$\bar{B} \cdot A$	Anything OR'ed with 0 is itself
$A \cdot \bar{B}$	Associative Law

Example

Simplify $\bar{A} \cdot \bar{B} \cdot \bar{C} + \bar{A} \cdot \bar{B} \cdot C + \bar{A} \cdot B \cdot \bar{C} + \bar{A} \cdot B \cdot C$

Solution

$\bar{A} \cdot (\bar{B} \cdot \bar{C} + \bar{B} \cdot C + B \cdot \bar{C} + B \cdot C)$	Distributive Law
$\bar{A} \cdot (\bar{B} \cdot (\bar{C} + C) + B \cdot (\bar{C} + C))$	Distributive Law
$\bar{A} \cdot (\bar{B} \cdot 1 + B \cdot 1)$	Anything OR'ed with its inverse is 1
$\bar{A} \cdot (\bar{B} + B)$	Anything AND'ed with 1 is itself
$\bar{A} \cdot 1$	Anything OR'ed with its inverse is 1
$\bar{A}$	Anything AND'ed with 1 is itself

5.7 DeMorgan's Theorem

Some of you may have noticed that the truth tables for the AND and OR gates are similar. Below is a comparison of the two operations.

AND			OR		
A	B	$X = A \cdot B$	A	B	$X = A + B$
0	0	0	0	0	0
0	1	0	0	1	1
1	0	0	1	0	1
1	1	1	1	1	1

2: Boolean Algebra

For the AND gate, the row that is different occurs when all of the inputs are ones, and for the OR gate, the different row occurs when all of the inputs are zeros. What would happen if we inverted the inputs of the AND truth table?

AND of inverted inputs

A	B	$X = \overline{A} \cdot \overline{B}$
0	0	1
0	1	0
1	0	0
1	1	0

OR with inverted output

A	B	$X = \overline{(A+B)}$
0	0	1
0	1	0
1	0	0
1	1	0

The two truth tables are still not quite the same, but they are quite close. The two truth tables are now inverses of one another.

AND of inverted inputs

A	B	$X = \overline{A} \cdot \overline{B}$
0	0	1
0	1	0
1	0	0
1	1	0

OR with inverted output

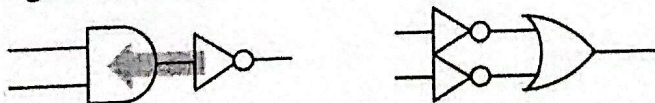
A	B	$X = \overline{(A+B)}$
0	0	1
0	1	0
1	0	0
1	1	0

So the output of an AND gate with inverted inputs is equal to the inverted output of an OR gate with non-inverted inputs. A similar proof can be used to show that the output of an OR gate with inverted inputs is equal to the inverted output of an AND gate with non-inverted inputs. This resolves our earlier discussion where we showed that the NOT gate cannot be distributed to the inputs of an AND or an OR gate. This brings us to DeMorgan's Theorem.

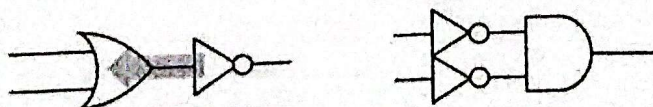
$$\overline{A+B} = \overline{A} \cdot \overline{B}$$

$$\overline{A \cdot B} = \overline{A} + \overline{B}$$

The purpose of DeMorgan's Theorem is to allow us to distribute an inverter from the output of an AND or OR gate to the gate's inputs. In doing so, an AND gate is switched to an OR gate and an OR gate is switched to an AND gate. The figure below shows how pushing the inverter from the output of a gate to its inputs affects the logic.



a.) Pushing an inverter through an AND gate flips it to an OR gate



b.) Pushing an inverter through an OR gate flips it to an AND gate

DeMorgan's Theorem applies to gates with three or more inputs too. One of the main purposes of DeMorgan's Theorem is to distribute any inverters in a digital circuit back to the inputs. This typically improves the circuit's performance by removing layers of logic and can be done either

2: Boolean Algebra

with the Boolean expression or the schematic. Either way, the inverters are pushed from the output side to the input side one gate at a time.

$$\begin{aligned}\overline{A + B + C + D} &= \overline{(A + B) \cdot (C + D)} \\ &= (\overline{A + B}) \cdot (\overline{C + D}) \\ &= \overline{A} \cdot \overline{B} \cdot \overline{C} \cdot \overline{D}\end{aligned}$$

Using the methods presented in here, it is understood that a Boolean expression can be manipulated into whatever form best suits the needs of the computer system. As far as the binary operation is concerned, two circuits are the same if their truth tables are equivalent. The circuits, however, may not be the same when measuring performance or when counting the number of gates it took to implement the circuit. The optimum circuit for a specific application can be designed using the tools presented in here.

Problems

1. List three drawbacks of using truth tables or schematics for describing a digital circuit.
2. List three benefits of a digital circuit that uses fewer gates.
3. True or False: Boolean expressions can be used to optimize the logic functions in software.
4. Convert the following Boolean expressions to their schematic equivalents. Do not modify the original expression

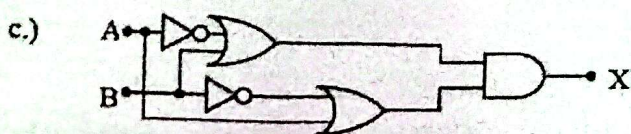
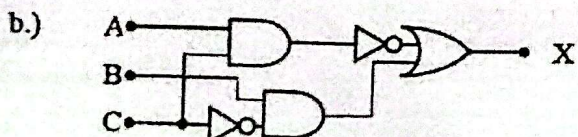
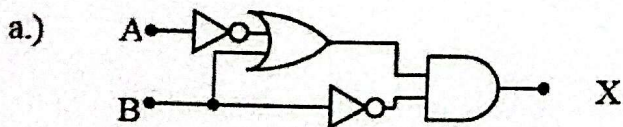
a.) $A \cdot B + C$

b.) $A \cdot \overline{B} \cdot C + \overline{A} \cdot B + \overline{A} \cdot C$

c.) $\overline{(A + B \cdot C)} + A \cdot \overline{D}$

d.) $A \cdot \overline{B} + \overline{A} \cdot B$

5. Convert each of the digital circuits shown below to their corresponding Boolean expressions without simplification.



6. Apply DeMorgan's Theorem to each of the following expressions so that the NOT bars do not

Depnyn... Nairobi 2025/12/07 22:41

2: Boolean Algebra

a.) $\overline{A \cdot C + B}$

b.) $\overline{D(C + B)(A + B)}$

c.) $\overline{A + \overline{B} + C + \overline{(AB)}}$

7. Simplify each of the following expressions.

a.) $B \cdot A + B \cdot \overline{A}$

b.) $(A + B)(B + \overline{A})$

c.) $A + \overline{B} + C + \overline{(AB)}$

d.) $\overline{B}(A + A \cdot B)$

e.) $(\overline{A} + B)(A + \overline{B})$

f.) $\overline{B} + \overline{(AB)} + \overline{C}$